

**PART XX****C++ OOP / UNIVERSITY MODULE**

*Classes, constructors, operator overloading, inheritance, polymorphism, and a first look at templates*

---

Everything in this Part exists because most university C++ courses are taught as Object-Oriented Programming courses, and this book promised full coverage of that syllabus alongside its competitive-programming core. Nothing here is needed for Codeforces or LeetCode — the algorithms and data structures Parts never assume you've read this one. Treat it as its own self-contained module: useful for exams, interviews that ask about OOP design, and understanding C++ codebases that use classes heavily, but deliberately kept out of the main problem-solving path of this book.

## Contents of Part XX

- 1. Why C++ Added OOP to C
- 2. Classes and Objects
- 3. Constructors and Destructors
- 4. Operator Overloading
- 5. Inheritance
- 6. Polymorphism
- 7. A First Look at Templates
- Part XX Revision, Cheat Sheet, and Interview Questions

# Chapter 1 — Why C++ Added OOP to C

## 1.1 C vs. C++, in one sentence

C organizes a program as data (structs) and functions that operate on that data, kept as two separate things; C++ adds the ability to bundle data and the functions that operate on it together into a single unit — a class.

“Conventional” (procedural) programming's disadvantage at scale: as a codebase grows, it becomes hard to guarantee that only the intended functions touch a given piece of data, because any function anywhere can reach into any struct. Classes let you restrict this on purpose — covered as “encapsulation” below.

## 1.2 The four pillars of OOP

| Pillar        | Meaning                                                                           | C++ mechanism                                         |
|---------------|-----------------------------------------------------------------------------------|-------------------------------------------------------|
| Encapsulation | Bundle data with the functions that operate on it, and hide internal details      | class, access specifiers ( private/protected/public ) |
| Abstraction   | Expose only what a user of the class needs to know, hide the how                  | public interface vs. private implementation           |
| Inheritance   | Build a new class from an existing one, reusing and extending its behavior        | class Derived : public Base                           |
| Polymorphism  | Let the same function call behave differently depending on the actual object type | virtual functions                                     |

## 1.3 Structure of a C++ program, and namespaces

```
#include <iostream>
using namespace std;

namespace MyMath { // namespaces prevent naming collisions
between libraries
    int square(int x) { return x * x; }
}

int main() {
    cout << MyMath::square(5) << endl; // :: is the scope resolution
operator
    return 0;
}
```

The scope resolution operator (::) says “look inside this specific scope for the following name” — used for namespaces above, and, as you'll see in Chapter 2, also used to define a class's member function outside the class body.

## Chapter 2 — Classes and Objects

In one sentence: a class is a blueprint that bundles data (member variables) and behavior (member functions) together; an object is one concrete instance created from that blueprint.

### 2.1 A minimal class

```
class Rectangle {
private:
    double width, height; // hidden from outside code --
    encapsulation

public:
    void setDimensions(double w, double h) { // member function
        width = w;
        height = h;
    }
    double area() const { // const: promises not to modify the object
        return width * height;
    }
};

int main() {
    Rectangle r; // r is an OBJECT -- an instance of class Rectangle
    r.setDimensions(3, 4);
    cout << r.area(); // 12
}
```

## 2.2 Access specifiers

| Specifier | Accessible from                                                          |
|-----------|--------------------------------------------------------------------------|
| private   | Only member functions of the same class (and friends — see 2.4)          |
| protected | Member functions of the same class, and of any derived class (Chapter 5) |
| public    | Anywhere the object is visible                                           |

class defaults every member to private if no specifier is given; struct defaults to public. This is the only fundamental difference between class and struct in C++ — everything else (inheritance, member functions, templates) works identically for both.

## 2.3 Inline functions and static members

```
class Counter {
    static int totalCreated; // shared across ALL objects of this
    class
public:
    Counter() { totalCreated++; }
    static int getTotalCreated() { return totalCreated; } // static
    member function
    inline int doubled(int x) { return x * 2; } // small function,
    hints the compiler to inline it
};
int Counter::totalCreated = 0; // static members need an
out-of-class definition

int main() {
    Counter a, b, c;
    cout << Counter::getTotalCreated(); // 3 -- called on the CLASS,
    not an object
}
```

A static member belongs to the class itself, not to any individual object — every object shares the exact same copy. This is the natural tool for “how many objects of this class currently exist,” shared configuration, or a shared counter/ID generator. `inline` is a hint (not a guarantee) that the compiler should substitute the function's body directly at each call site instead of performing a normal function call, saving call overhead for very small functions; modern compilers usually make this decision on their own regardless of the keyword.

## 2.4 Friend functions

A friend function is a non-member function granted access to a class's private members — an explicit, deliberate exception to encapsulation, used sparingly (most commonly for operator overloading, Chapter 4).

```
class Box {
    double volume;
public:
    Box(double v) : volume(v) {}
    friend void printVolume(const Box &b); // declared inside,
defined outside
};
void printVolume(const Box &b) {
    cout << b.volume; // allowed: printVolume is a friend, even
though volume is private
}
```

## 2.5 Function overloading (as a class member)

Introduced in Part I for free functions — works identically for member functions: the same function name, different parameter lists.

```
class Printer {
public:
    void print(int x) { cout << "int: " << x; }
    void print(double x) { cout << "double: " << x; }
    void print(string s) { cout << "string: " << s; }
};
```

## 2.6 Array of objects, object as parameter, object as return value

```
Rectangle rects[3]; // array of objects -- each
default-constructed
rects[0].setDimensions(2, 3);

double totalArea(Rectangle r) { // object passed BY VALUE -- a
full copy is made
    return r.area();
}

double totalAreaFast(const Rectangle &r) { // by const reference
-- no copy, no modification
    return r.area();
}

Rectangle makeUnitSquare() { // object returned BY VALUE
    Rectangle r;
    r.setDimensions(1, 1);
    return r;
}
```

### ⚠ Common mistake

Passing large objects by value into functions repeatedly, which silently copies the entire object every call. Exactly like Part I's guidance for vectors and strings, pass by const reference unless the function genuinely needs to own a separate copy.

## 2.7 Nested classes

```
class LinkedList {
    struct Node { // nested class/struct -- an implementation detail
of LinkedList
        int value;
        Node *next;
    };
    Node *head = nullptr;
public:
    void pushFront(int v) {
        Node *n = new Node{v, head};
        head = n;
    }
};
```

Nesting Node inside LinkedList communicates that Node is purely an implementation detail of LinkedList, not a type meant to be used independently — this is the same Node concept used throughout Part VIII (Linked Lists), here shown as a formally nested class.

## Practice — Chapter 2

### Problem 20.1 — Design a Bank Account Class [Level 1]

Hint 1: Balance should be private; deposit/withdraw should be the only ways to change it.

Approach: Encapsulate balance as private; provide public deposit(amount) and withdraw(amount) methods that validate the amount before changing balance.

## Chapter 3 — Constructors and Destructors

In one sentence: a constructor runs automatically when an object is created, to set it up in a valid initial state; a destructor runs automatically when an object is destroyed, to clean up any resources it holds.

### 3.1 Constructor overloading and the initializer list

```
class Point {
    double x, y;
public:
    Point() : x(0), y(0) {} // default constructor
    Point(double x, double y) : x(x), y(y) {} // parameterized
    constructor
    void print() const { cout << "(" << x << ", " << y << ")"; }
};

int main() {
    Point origin; // calls the default constructor
    Point p(3, 4); // calls the parameterized constructor --
    overload resolution
}
```

The initializer list (: x(0), y(0)) initializes members directly, before the constructor's body runs. This matters, not just style: for const members and reference members, direct initialization via the initializer list is the **ONLY** option — they cannot be assigned inside the body, since that would be a second step after an already-completed (and, for const/reference, impossible) default initialization.

## 3.2 The copy constructor

A copy constructor builds a new object as a copy of an existing one. The compiler generates one automatically (copying each member) unless you define your own — which becomes necessary the moment a class manages a resource like raw dynamic memory.

```
class Buffer {
    int *data;
    int size;
public:
    Buffer(int n) : size(n) { data = new int[n]; }
    Buffer(const Buffer &other) : size(other.size) { // copy
    constructor
        data = new int[size]; // allocate a SEPARATE array
        for (int i = 0; i < size; i++) data[i] = other.data[i];
    }
    ~Buffer() { delete[] data; } // destructor
};
```

**⚠ Why the copy constructor must deep-copy here**

Without a custom copy constructor, the compiler-generated default would copy the data pointer itself (a shallow copy) — leaving both objects pointing at the SAME array. When either object is destroyed, its destructor calls `delete[]` on that shared array; the other object is left holding a dangling pointer, and a second `delete[]` on the same memory is undefined behaviour (a double-free). This exact bug is one of the most common sources of crashes in classes that manage raw pointers.

### 3.3 Anonymous objects

```
double area = Rectangle(3, 4).area(); // Rectangle(3, 4) is an
ANONYMOUS object --
// created, used, and destroyed in one expression
```

### 3.4 Destructor order and the this pointer

Objects are destroyed in the reverse order of their construction (last constructed, first destroyed) — relevant for stack-allocated objects going out of scope, and for base/derived destructor order (Chapter 5). Inside any member function, `this` is a pointer to the object the function was called on — used to disambiguate a member from a same-named parameter, or to return the object itself for method chaining.

```
class Builder {
    string result;
public:
    Builder& add(string s) { // returns *this to allow chaining
        result += s;
        return *this;
    }
    string get() const { return result; }
};
// Builder().add("Hello, ").add("World!").get();
```

## Practice — Chapter 3

## Problem 20.2 — Fix the Shallow Copy Bug [Level 2]

Hint 1: A class holds a raw `int*` and has no user-defined copy constructor.

Key observation: Copying an object with the default (shallow) copy constructor duplicates the pointer, not the array it points to.

Approach: Write a copy constructor that allocates a new array and copies the pointed-to values, and a matching destructor that frees it.

# Chapter 4 — Operator Overloading

In one sentence: operator overloading lets you define what a built-in operator (`+`, `==`, `<<`, etc.) means when applied to your own class's objects.

## 4.1 The operator keyword, and general rules

- You cannot invent new operators or change an operator's number of operands (unary stays unary, binary stays binary).
- At least one operand must be a user-defined type — you cannot redefine what `+` means for two built-in ints.
- Operator precedence and associativity cannot be changed, only what the operator computes for your type.

## 4.2 Binary operator overloading — member-function approach

```
class Vector2D {
public:
    double x, y;
    Vector2D(double x, double y) : x(x), y(y) {}

    Vector2D operator+(const Vector2D &other) const {
        return Vector2D(x + other.x, y + other.y);
    }
    bool operator==(const Vector2D &other) const {
        return x == other.x && y == other.y;
    }
};
// usage: Vector2D c = a + b; is really a.operator+(b);
```

## 4.3 Binary operator overloading — friend-function approach

When the LEFT operand isn't your class type (e.g. `cout << myObject`, where the left operand is `std::ostream`, a type you can't add member functions to), a member function can't be the left operand's method — use a friend function instead.

```
class Vector2D {
public:
    double x, y;
    Vector2D(double x, double y) : x(x), y(y) {}
    friend ostream& operator<<(ostream &out, const Vector2D &v);
};
ostream& operator<<(ostream &out, const Vector2D &v) {
    out << "(" << v.x << ", " << v.y << ")";
    return out; // return the stream so << can be chained: cout << a
<< b;
}
```

## 4.4 Unary operator overloading

```
class Counter {
    int value;
public:
    Counter(int v) : value(v) {}
    Counter& operator++() { // prefix ++counter
        value++;
        return *this;
    }
    Counter operator++(int) { // postfix counter++ -- the unused '
int' marks postfix
        Counter temp = *this;
        value++;
        return temp;
    }
};
```

#### ♦ How to choose member vs. friend

Use a member function when your class is naturally the LEFT operand (a + b where a is your type). Use a friend function when the left operand is a built-in type or a type you don't control (most commonly operator<< and operator>> for stream I/O).

## Practice — Chapter 4

### Problem 20.3 — Overload Operators for a Fraction Class [Level 2]

Hint 1: You'll need +, ==, and operator<< at minimum for a usable Fraction class.

Approach: Implement operator+ as a member (Fraction + Fraction), and operator<< as a friend function since cout is the left operand.

## Chapter 5 — Inheritance

In one sentence: inheritance lets a new class (the derived class) reuse the data and behavior of an existing class (the base class), adding or overriding only what's different.

```
class Animal {
protected:
    string name;
public:
    Animal(string n) : name(n) {}
    void eat() { cout << name << " is eating\n"; }
};

class Dog : public Animal { // Dog IS-A Animal
public:
    Dog(string n) : Animal(n) {} // must explicitly call the base
    constructor
    void bark() { cout << name << " says woof\n"; } // name is
    accessible: protected
};

int main() {
    Dog d("Rex");
    d.eat(); // inherited from Animal
    d.bark(); // Dog's own method
}
```

## 5.1 Types of inheritance

| Type         | Shape                                                                       |
|--------------|-----------------------------------------------------------------------------|
| Single       | One base, one derived (the Dog example above)                               |
| Multilevel   | A chain: Grandparent → Parent → Child, each inheriting from the previous    |
| Hierarchical | One base, multiple independent derived classes (Animal → Dog, Animal → Cat) |
| Multiple     | One derived class inherits from more than one base class directly           |
| Hybrid       | A combination of the above shapes in one design                             |

## 5.2 Multiple inheritance and the diamond problem

```

class A { public: int x = 1; };
class B : public A {};
class C : public A {};
class D : public B, public C {}; // D has TWO copies of A's data
-- ambiguous!

// D d; d.x; // COMPILE ERROR: ambiguous, which A::x?

// Fix: virtual inheritance -- B and C share a single A
class A2 { public: int x = 1; };
class B2 : virtual public A2 {};
class C2 : virtual public A2 {};
class D2 : public B2, public C2 {}; // D2 has exactly ONE A2
subobject
// D2 d2; d2.x; // fine -- unambiguous

```

This is the classic “diamond problem”: without virtual inheritance, D inherits A twice (once through B, once through C), so any reference to a member of A is ambiguous. virtual base classes solve this by guaranteeing only one shared copy of A exists no matter how many paths lead to it.

## 5.3 this pointer, and base/derived pointers

```

Animal *a = new Dog("Rex"); // a Dog, accessed through an Animal
pointer
a->eat(); // fine -- eat() is Animal's, always resolves the same
way
// a->bark(); // COMPILE ERROR: Animal has no bark() -- the
pointer's
// declared type limits what's visible without a cast

Dog *d = dynamic_cast<Dog*>(a); // safely re-obtain the derived
type, if it really is one
if (d) d->bark();

```

A base-class pointer can point at a derived object (this is exactly what polymorphism, Chapter 6, relies on) — but through that pointer, only members declared in the base class's own interface are directly callable, unless the function is virtual. `dynamic_cast` safely attempts to “downcast” back to a more derived type, returning `nullptr` if the object isn't actually of that type.

## 5.4 Object as a class member (composition)

```
class Engine {
public:
    void start() { cout << "Engine started\n"; }
};
class Car {
    Engine engine; // Car HAS-A Engine -- composition, not
inheritance
public:
    void drive() { engine.start(); cout << "Car driving\n"; }
};
```

### ✦ Inheritance (IS-A) vs. composition (HAS-A)

A common design question: should Y inherit from X, or should Y simply hold an X as a member? Use inheritance when Y genuinely IS a more specific kind of X (Dog IS-A Animal); use composition when Y merely USES or CONTAINS an X (Car HAS-A Engine — a car is not a kind of engine). Overusing inheritance where composition fits better is a very common design mistake in introductory OOP code.

## Practice — Chapter 5

### Problem 20.4 — Shape Hierarchy [Level 2]

Hint 1: Circle, Rectangle, and Triangle are all specific kinds of Shape.

Approach: Design a base Shape class with common data (e.g. a name), and derive Circle/Rectangle/Triangle from it, each adding their own dimension fields.

## Chapter 6 — Polymorphism

In one sentence: polymorphism lets the same function call, written once against a base-class type, automatically run the correct derived class's version of that function — decided by the object's actual type at runtime, not by the pointer/reference's declared type.

### 6.1 Early binding vs. late binding

|              | Early (static) binding                                      | Late (dynamic) binding                              |
|--------------|-------------------------------------------------------------|-----------------------------------------------------|
| When decided | At compile time                                             | At runtime                                          |
| Mechanism    | Ordinary (non-virtual) function calls, function overloading | virtual functions                                   |
| Cost         | None — resolved directly                                    | Small runtime lookup (a vtable pointer indirection) |

## 6.2 Virtual functions

```

class Animal {
public:
    virtual void speak() { cout << "Some generic animal sound\n"; }
    virtual ~Animal() {} // virtual destructor -- see 6.4
};
class Dog : public Animal {
public:
    void speak() override { cout << "Woof!\n"; } // override is
optional but recommended
};
class Cat : public Animal {
public:
    void speak() override { cout << "Meow!\n"; }
};

void makeItSpeak(Animal *a) {
    a->speak(); // calls Dog's or Cat's version, chosen at RUNTIME
}
// makeItSpeak(new Dog()); -> "Woof!"
// makeItSpeak(new Cat()); -> "Meow!"

```

Without virtual, `a->speak()` would always call `Animal::speak()` no matter what `a` actually points to — that's early binding. Marking `speak()` virtual switches to late binding: the call is resolved by looking up the object's actual type at runtime. The `override` keyword doesn't change behavior, but it makes the compiler verify you're actually overriding a real virtual function — catching typos (a mismatched signature) as a compile error instead of a silent bug.

## 6.3 Pure virtual functions and abstract classes

```
class Shape {
public:
    virtual double area() const = 0; // PURE virtual -- no
    implementation here
    virtual ~Shape() {}
};
// Shape s; // COMPILE ERROR -- Shape is abstract, cannot be
// instantiated

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius;
}
};
```

A class with at least one pure virtual function is an abstract class — it cannot be instantiated directly, and exists purely to define an interface that derived classes must implement. This is how C++ expresses “every Shape must provide an area() calculation, but there's no such thing as a generic Shape object on its own.”

## 6.4 Why the destructor should be virtual

### ⚠ Concrete failure without a virtual destructor

Given `Animal *a = new Dog(); delete a;` — if Animal's destructor is NOT virtual, only Animal's destructor runs, even though the object is really a Dog. Any resources Dog's destructor was supposed to free (e.g. dynamically allocated members) leak. Marking the base class's destructor virtual ensures delete through a base pointer correctly calls the full derived-class destructor chain, most-derived first.

Practical rule: if a class has ANY virtual function, give it a virtual destructor too — the presence of virtual functions signals the class is meant to be used polymorphically through base pointers, which is exactly the situation this rule protects.

## Practice — Chapter 6

## Problem 20.5 — Polymorphic Shape Area Calculator [Level 2]

Hint 1: Store a vector of base-class pointers (or smart pointers) to different derived shapes.

Approach: Define an abstract Shape base with pure virtual area(); implement Circle, Rectangle, Triangle; sum area() across a vector to get the total, relying on virtual dispatch.

## Chapter 7 — A First Look at Templates

In one sentence: a template lets you write one function or class definition that works for many different types, with the compiler generating the specific version needed for each type actually used.

### 7.1 Function templates

```
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << maxOf(3, 7); // T deduced as int
    cout << maxOf(2.5, 1.1); // T deduced as double
    cout << maxOf(string("abc"), string("abd")); // T deduced as
    string
}
```

This is exactly why `std::max`, `std::sort`, and every other STL algorithm from Part IV work across so many types without the library authors writing a separate version for each one — they're all templates.

### 7.2 Class templates

```
template <typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() const { return value; }
};

int main() {
    Box<int> intBox(42);
    Box<string> strBox("hello");
}
```

`vector<T>`, `pair<T1,T2>`, and every other STL container from Part IV are class templates — the angle-bracket syntax you've been using throughout this book is exactly this mechanism.

#### **Scope note**

This chapter is intentionally brief. Function-template overloading, template specialization, and templates-vs-macros — along with a full templated linked list — are covered in depth in the next Part, alongside exceptions and file I/O, so that all three “additional C++ mechanisms” topics sit together.

# Part XX — Master Revision

## Must Remember

- The four pillars of OOP: encapsulation, abstraction, inheritance, polymorphism.
- class defaults to private members; struct defaults to public — otherwise identical.
- A custom copy constructor (and destructor) is required the moment a class manages a raw resource like dynamic memory, to avoid a shallow-copy double-free.
- Operator overloading: use a member function when your class is the left operand; use a friend function when it isn't (most commonly `operator<<()`).
- Multiple inheritance's diamond problem is solved with virtual base classes, guaranteeing a single shared subobject.
- IS-A relationships use inheritance; HAS-A relationships use composition — don't default to inheritance when composition fits better.
- virtual functions enable late (runtime) binding; without virtual, calls through a base pointer always resolve to the base class's version.
- A pure virtual function (`= 0`) makes a class abstract — it cannot be instantiated and exists to define an interface.
- Any class with virtual functions should have a virtual destructor, or deleting a derived object through a base pointer leaks resources.
- Templates let one function/class definition work across many types — this is exactly how the STL itself is built.

## Common Mistakes Recap

| Mistake                                                                              | Fix                                                             |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Relying on the compiler-generated copy constructor for a class holding a raw pointer | Write a deep-copying copy constructor and a matching destructor |

| Mistake                                                                          | Fix                                                                      |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Forgetting the base class constructor call in a derived class's initializer list | Explicitly call Base(args) in the derived constructor's initializer list |
| Multiple inheritance without virtual base classes when a shared ancestor exists  | Use virtual public Base to avoid the diamond-problem ambiguity           |
| A non-virtual destructor in a class meant to be used polymorphically             | Mark the destructor virtual whenever the class has any virtual function  |
| Passing large objects by value into member/free functions repeatedly             | Pass by const reference unless a genuine independent copy is needed      |

## Interview-Style Questions

- 1 What are the four pillars of OOP, and what C++ mechanism implements each one?
- 2 Why does a class managing raw dynamic memory need a custom copy constructor and destructor?
- 3 When should you overload an operator as a member function versus as a friend function?
- 4 What is the diamond problem in multiple inheritance, and how do virtual base classes solve it?
- 5 Why does a base class need a virtual destructor if it has any virtual functions?
- 6 What is the difference between early binding and late binding in C++?
- 7 Why can't an abstract class be instantiated, and what is it useful for instead?

## Cheat Sheet — Part XX

```
class C {
private: /* hidden */ protected: /* class + derived */ public: /*
everyone */
public:
    C() {} // default constructor
    C(const C &other) {} // copy constructor -- deep copy if
managing resources
    ~C() {} // destructor -- virtual if base of polymorphic hierarchy
    C& operator=(const C &o) { return *this; } // copy assignment
    virtual void f() {} // late-bound (overridable) function
    virtual void g() = 0; // pure virtual -- makes C abstract
};
class D : public C { void f() override {} void g() override {} };

template <typename T> T myMax(T a, T b) { return a > b ? a : b; }
template <typename T> class Box { T v; public: Box(T v): v(v) {}
T get(){return v;} };
```

### **Coming up in Part XXI**

Part XXI — Exceptions, Files and Templates finishes the remaining “additional C++ mechanisms”: try/throw/catch and exception hierarchies, reading and writing files with stream I/O, and a full return to templates — function-template overloading, template specialization, templates vs. macros, and a complete templated linked list.